The Big GEM Theory
Takneek PS -Zenith Part E: Documentation & Reports

Scope: *Part 1: Mathematical definition of the modified scheduling equations. Part 2: Architecture block diagrams mapping DSP48E2 / CARRY4 / SRLC32E to the GPU memory hierarchy. Part 3: Throughput analysis (analytical model, measurement protocol, results).*

Table 1: Notation Reference

Notation	Meaning
$G = (V, E)$	Unified heterogeneous dependency graph for one simulated clock cycle
R, M	AIG-region vertices and macro-instance vertices; $V = R \cup M$
$E_{\textbf{same}}$	Same-cycle combinational edges (contribute to level recurrence)
$E_{\textbf{next}}$	Edges crossing the clock boundary (excluded from level recurrence)
S	Set of value sources available at cycle start
$\textbf{level}(v)$	Dependency level (wave index) assigned to vertex v
L	Number of levels, $L = 1 + \max_v \text{level}(v)$
$\textbf{cut}(g)$	Macro cut of AND-gate pin g

1. Part 1 — Mathematical Definition of the Modified Scheduling Equations

GEM's unmodified scheduler levelizes a pure 1-bit And-Inverter Graph (AIG): every vertex is an AND gate, every edge is a wire. The modification adds word-level macro vertices and a second class of edge, levelizes the combined graph, and keeps stretches of AND gates fused into regions so the schedule has one barrier per dependency depth, not one per gate.

1.1 Vertices

$$V = R \cup M, \quad M = M_{\text{carry}} \cup M_{\text{dsp}} \cup M_{\text{srl}}$$

M contains exactly one vertex per preserved macro instance. R contains the fused AIG regions defined in Section 1.2. Every synthesised operation maps to exactly one vertex, and every vertex carries exactly one kind $k(v) \in \{\text{AigRegion}, \text{Carry4}, \text{Dsp48e2}, \text{Srlc32e}\}$ (the single-producer, single-occurrence invariant).

1.2 Fused AIG Regions (The Macro Cut)

Placing every AND gate in its own schedule level would insert one wave barrier per gate. Instead, the gates are fused into regions — for each AND-gate pin g with fan-in pins x_1, x_2:

$$\text{cut}(g) = \text{contrib}(x_1) \cup \text{contrib}(x_2)$$

$$\text{contrib}(x) = \begin{cases} \text{cut}(x) & \text{if driver}(x) \text{ is an AND gate} \\ \{\text{node}(x)\} & \text{if driver}(x) \text{ is a CARRY4 or SRLC32E output pin} \\ \emptyset & \text{if } x \text{ is a source or a DSP48E2 } P \text{ pin} \end{cases}$$

cut is monotone non-decreasing along AND edges, so a single pass in ascending pin-id order (which is a topological order by AIG construction) computes it. A region is an equivalence class of AND-gate pins under cut-equality:

$$g \sim g' \iff \text{cut}(g) = \text{cut}(g'); \quad R = (\text{AND-gate pins})/\sim$$

A region depends on exactly the macros in its (shared) cut, and every one of them is strictly upstream, so a region-macro cycle in G is impossible unless the netlist contains a genuine combinational loop.

1.3 Value Sources and the Edge Partition

$$S = \text{PI} \cup Q_{\text{dff}} \cup \text{RD}_{\text{sram}} \cup P_{\text{dsp}} \cup \{0\}$$

PI are primary inputs; Q_{dff}, RD_{sram}, and P_{dsp} are the outputs of clocked elements. P_{dsp} is in S because the specification clocks only `PREG`: a same-cycle reader of a `DSP48E2` P output observes the value latched on the previous edge, identical to a flip-flop output. A net produced this cycle by a vertex u makes u a same-cycle producer; a net in S has no producer.

Same-Cycle Edges (E_{same}): $(u, v) \in E_{\text{same}}$ iff v consumes, in the current cycle, a net produced by u in the current cycle (Table 2).

Table 2: Realising Conditions for Same-Cycle Edges E_{same}

u	v	Realising Condition
Region r	Macro m	A gate of r drives a data pin of m (`S/DI/CIN/CYINIT` for `CARRY4`; `D/CE/A` for `SRLC32E`; `A/B/C/D/OPMODE/ALUMODE/INMODE/CEP/RSTP` for `DSP48E2`)
Macro m	Region r	A combinational output pin of m (`CARRY4` $O[i]$/CO$[i]$, `SRLC32E` $Q/Q31$) feeds a gate of r
Macro m	Macro m'	A combinational output pin of m directly drives a data pin of m' (the CO[3] $\to$ CIN case) — realised as one direct edge, no boolean node
Region r	Region r'	A gate of r feeds a gate of r' and $\text{cut}(r) \neq \text{cut}(r')$

`DSP48E2` contributes no outgoing E_{same} edge (P is registered). `SRLC32E` does: the frozen semantics evaluate the asynchronous $Q/Q31$ taps against post-shift storage within the same timestamp.

Cross-Cycle Edges (E_{next}):

$E_{\text{next}} = \{D \to Q : \text{DFF}\} \cup \{P_{\text{next}} \to P : \text{DSP48E2}\} \cup \{\text{shift\_next} \to \text{storage} : \text{SRLC32E}\} \cup \{\text{wr} \to \text{rd} : \text{SRAM}$

E_{next} is excluded from the same-cycle in-degree. Consequently, registered feedback is always schedulable, and the only topology the scheduler rejects is a real combinational loop.

1.4 Levelisation

$$\text{level}(v) = \begin{cases} 0 & \text{if } \{u : (u,v) \in E_{\text{same}}\} = \emptyset \\ 1 + \max\{\text{level}(u) : (u,v) \in E_{\text{same}}\} & \text{otherwise} \end{cases}$$

Computed by Kahn's algorithm over E_{same}: repeatedly remove a zero-in-degree vertex and relax its out-edges. Let the algorithm remove N vertices in total.

Table 3: Properties and Invariants of Levelisation

Property	Statement
Termination / Soundness	If $N < \|V\|$, the residual sub-graph has a directed cycle in E_{same}; the schedule is rejected and every residual vertex is named. If $N = \|V\|$, the level assignment is total.
Invariant P1	For every $(u,v) \in E_{\text{same}}$: $\text{level}(u) < \text{level}(v)$ (strict).
Invariant P2	Every vertex appears in exactly one level.
Part B Guarantee	For every macro-to-macro dependency d realised on a single net n with driver(n) a `CARRY4`/`SRLC32E` output (i.e. d.direct): level(producer(d)) < level(consumer(d)). Enforced by `verify_topological_guarantee()`; a schedule that would violate it does not build.

Proof of P1. Kahn removes u before every successor; $\text{level}(u)$ is final when u is removed; each relaxation sets $\text{level}(v) \geq \text{level}(u) + 1$.

Proof of P2. level is a well-defined function on V. Rejection completeness: a vertex is removed iff its in-degree reaches zero; a vertex on a directed cycle never does; therefore all and only cycle-reachable vertices remain. ■

1.5 Type-Homogeneous Wave Decomposition

$$\text{wave}(l) = \{v \in V : \text{level}(v) = l\}$$

$$\text{Wave}(l) = Q_{\text{aig}}(l) \sqcup Q_{\text{carry}}(l) \sqcup Q_{\text{dsp}}(l) \sqcup Q_{\text{srl}}(l)$$

$$Q_k(l) = \{v \in \text{wave}(l) : k(v) = k\}$$

Each queue holds a single primitive kind, so a GPU warp draining a queue never executes a per-instance branch over primitive types.

1.6 Execution Schedule and the Commit Rule

```
state_old : persistent image at cycle start
arena := 0; stage the gathered state words into arena (immutable for
   cycle)

for l = 0 .. L-1:
    evaluate Q_aig(l)   region gates by internal dependency depth,
                        parallel across block, barrier between depths
    evaluate Q_carry(l) one instance per lane, warp-stride
```

```
    evaluate Q_srl(l)   one instance per lane
    evaluate Q_dsp(l)   one instance per lane, 64-bit ALU
    publish level l:    __syncwarp / __syncthreads (stage to global
       grid.sync)

commit once: { D->Q, P_next->P, shift_next->storage, SRAM wr } ->
   state_next
```

Listing 1: Execution Schedule Pseudocode

The commit rule (each persistent word written exactly once, at the cycle boundary) gives the old-state-immutability property that makes the level loop race-free.

1.7 Complexity Analysis

Table 4: Algorithmic Complexity Bounds

Quantity	Complexity Bound
Cut computation	$\mathcal{O}$(AND gates) time, $\mathcal{O}$(AND gates $\times$ avg_cut_size) space
Kahn levelisation	$\mathcal{O}(\lvert V\rvert + \lvert E_{\text{same}}\rvert)$
Wave decomposition	$\mathcal{O}(\lvert V\rvert)$
Barriers per cycle	L block-level, plus 1 grid-level per major-stage boundary (vs. depth(AIG) in the shred-everything baseline)
Dependency proofs	$\mathcal{O}\left(\sum_{\text{macros}} \text{input\_pins}\right)$

1.8 Worked Example: The CO[3] $\rightarrow$ CIN Cascade

Eight `CARRY4`s, each CO[3] wired to the next CIN; S and DI tied to a shared 4-bit input.

- resolve : driver(CIN$_k$) = CARRY4(cell$_{k-1}$, 7), a direct macro net.
- $E_{\text{same}} = \{(m_0, m_1), (m_1, m_2), \ldots, (m_6, m_7)\}$.
- Kahn gives level $l(m_k) = k$, so $L = 8$ and Wave$(k) = Q_{\text{carry}}(k) = \{m_k\}$.
- `macro_dependencies()` returns 7 proofs, each (producer CO[3], consumer CIN, direct = true, producer_wave = k, consumer_wave = $k + 1$).
- The reference interpreter reproduces plain sequential carry propagation for all 16 input vectors (`test_carry_cascade_8_deep_sequence_matches_sequential_reference`).

2. Part 2 — FPGA Primitives Mapped to the GPU Memory Hierarchy

2.1 The Three Tiers

Design rule: A datum lives at the highest (fastest) tier that all of its consumers can reach. Thread-private values never leave registers; values shared between waves or warps go to shared memory; only persistent state and the read-only program touch global memory, moved in one coalesced pass per cycle.

Table 5: GPU Memory Hierarchy Allocation

Tier	Location / Scope	Latency	Size	Holds (in this design)
Registers	Per thread, private	$\sim$ 1 cycle	255×32-bit	Gathered operands, arithmetic temporaries (AD, M, carry chain, shifted state), result words
Shared	On-chip, per block	$\sim$ 20–30 cyc	48–100 KB	`u64` value arena (staged state + current-cycle macro and region-gate outputs)
Global (VRAM)	Off-chip, whole grid	$\sim$ 400–600 cyc	GBs	Old-state and next-state images, immutable V2 program (uploaded once), SRAM storage

2.1.1 Evaluation GPU Comparison (NVIDIA RTX 5060 vs GTX 1650)

2.2 Overall Dataflow

Figure 1 illustrates the dataflow across the GPU memory hierarchy during a single simulated cycle.

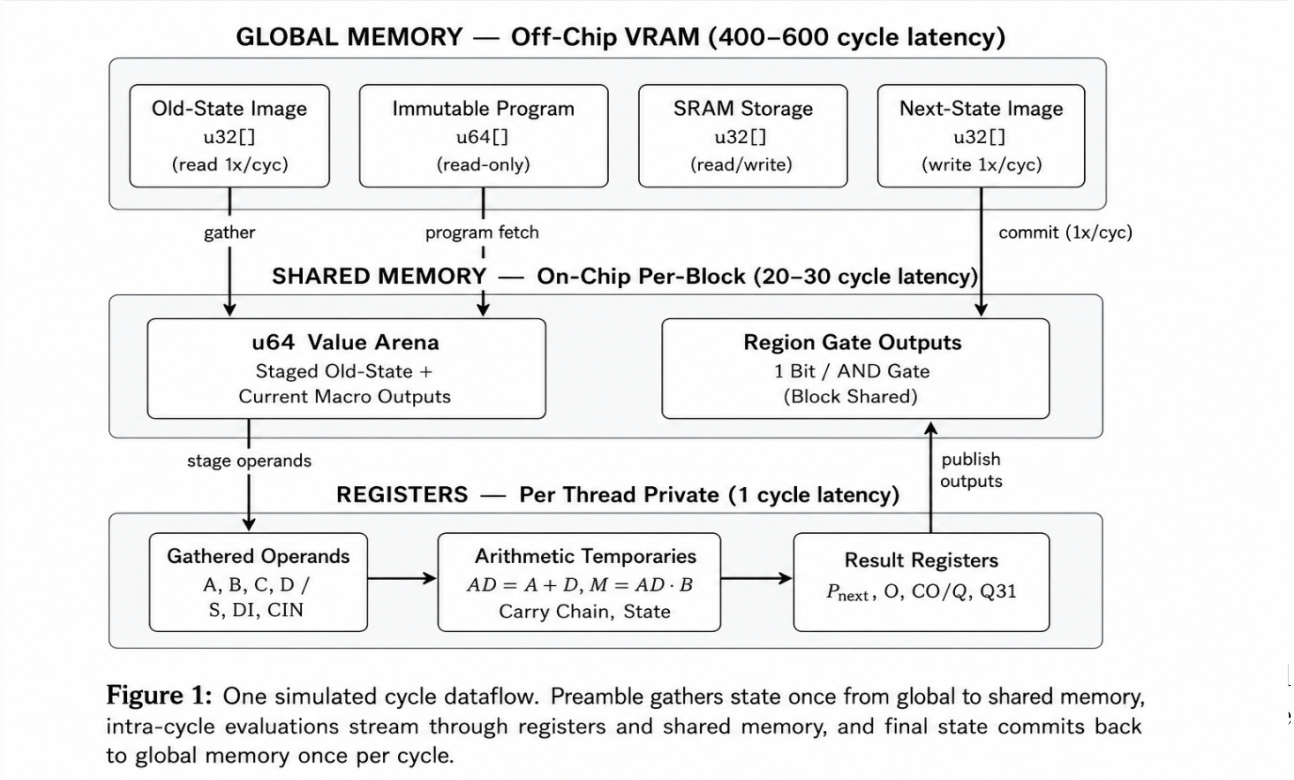


Figure 1: One simulated cycle dataflow. Preamble gathers state once from global to shared memory, intra-cycle evaluations stream through registers and shared memory, and final state commits back to global memory once per cycle.

Table 6: Target GPU Specifications and Kernel Mapping

Resource	RTX 5060 (CC 12.0)	GTX 1650 (CC 7.5)	How V2 Kernel Uses It
SMs	~ 30	14	One cooperative block per SM (grid = SM count), clamped via `cudaOccupancyMaxActiveBlocksPe`
Regs / SM	$65,536 \times 32$-bit	$65,536 \times 32$-bit	Capped at 128 regs/thread (`-maxrregcount=128`); operands, temporaries, results fit
Shared / SM	Up to 100 KB	64 KB	Per-block `u64` value arena + region-gate output bits; sized from schedule
L2 Cache	~ 32 MB	1 MB	Immutable V2 program and state images stream through L2; field-major layout retains 128B usage
VRAM	8 GB GDDR7	4 GB GDDR6	State images (u32[cycles × state_words]) and SRAM storage

2.3 DSP48E2 Mapping

Figure 2 demonstrates the dataflow for the `DSP48E2` multiply-accumulate macro.

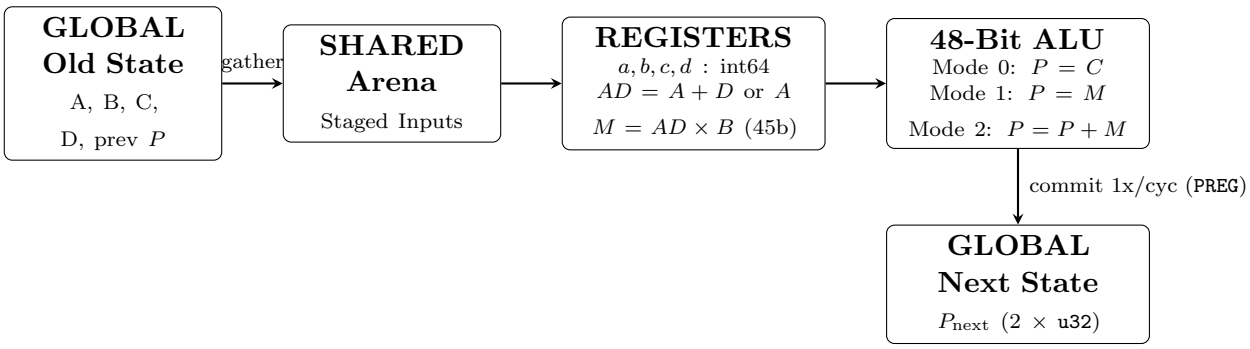


Figure 2: DSP48E2 mapping. `PREG` is the only clocked register that reaches global persistent state. All intermediate registers remain in thread-private GPU registers.

2.4 CARRY4 Mapping

Figure 3 illustrates the execution flow of the `CARRY4` primitive.

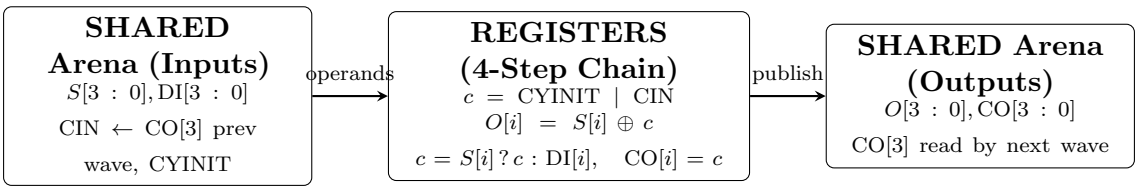
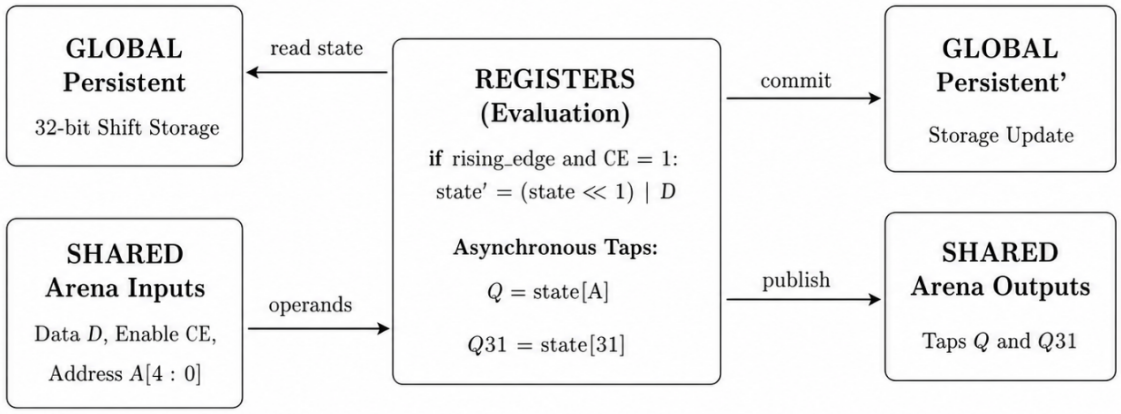


Figure 3: Purely combinational carry chain. Memory traffic is limited to 10 input bits read from shared arena and 8 output bits written back to it.

2.5 SRLC32E Mapping

Figure 4 maps the SRLC32E shift register execution model.



2.6 Wave-Execution Timeline and Barrier Taxonomy

Figure 5 details the barrier timeline across simulated cycle waves.

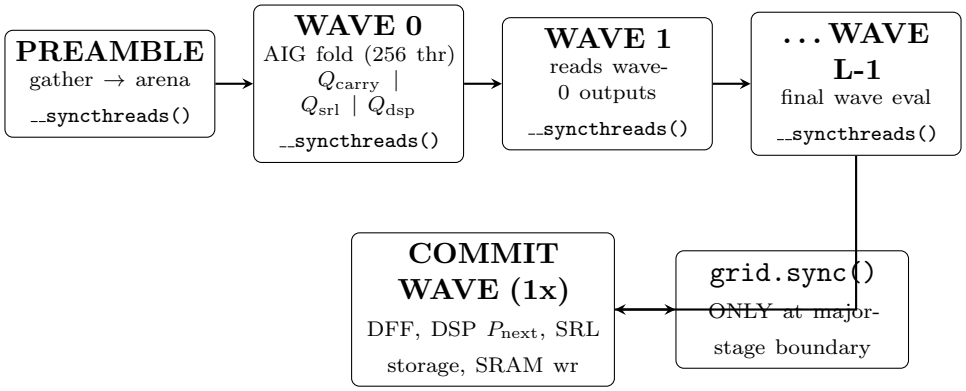


Figure 5: Wave timeline. Barriers used: `__syncwarp(mask)` for warp-local operations, `__syncthreads()` for block-local boundaries, and `grid.sync()` for inter-block synchronization.

2.7 Buffer Placement Rationale

- **Immutable program in Global (uploaded once):** Read-only and identical across cycles; amortises transfer overhead. Field/bit-major layout ensures warp reads are fully coalesced.

- `u64` **value arena in Shared:** Same-cycle macro outputs consumed in the same block are shared at minimal latency.
- **Operands/temporaries in Registers:** Thread-private state (AD, M, carry chain) kept in registers avoids shared memory bank conflicts and maintains peak execution rate.
- **Persistent state in Global (committed once):** Writing state at the cycle boundary ensures cycle immutability during evaluation and prevents race conditions.

3. Part 3 — Throughput Analysis

3.1 Cost of Shredding a Macro

Table 7: Estimated AIG Expansion Impact per Macro Primitive

Macro Primitive	AIG Nodes After Mapping	Critical-Path AIG Levels (Est.)	Native Representation
`DSP48E2` (27×18 signed MAC)	2,000–5,000	30–60	1 node, sink at its wave
`CARRY4`	20–40	8–12 (ripple)	1 node
`SRLC32E` ($32:1$ mux + shift)	90–150 (mux) + 32 DFFs	5–6	1 node

3.2 Execution-Time Model

Let L_{aig} be the non-macro AIG critical-path levels, n_{chain} the longest same-cycle macro chain, d_{macro} the mean shredded macro depth, t_{wave} per-wave macro cost, and b per-barrier cost:

$$T_{\text{shred}} \propto c_{\text{fold}} \times (L_{\text{aig}} + n_{\text{chain}} \times d_{\text{macro}})$$

$$T_{\text{native}} \propto c_{\text{fold}} \times L_{\text{aig}} + n_{\text{chain}} \times (t_{\text{wave}} + b)$$

$$\text{Speedup } S = \frac{T_{\text{shred}}}{T_{\text{native}}}$$

For macro-dominated designs (L_{aig} small, $n_{\text{chain}} d_{\text{macro}}$ large), $S \to d_{\text{macro}}/(t_{\text{wave}} + b)$, removing the shredded macro depth from the critical path.

3.3 Memory-Traffic Model

Table 8: Memory Traffic Comparison per Macro per Cycle

Primitive	Shredded Traffic (Bytes/Macro/Cycle)	Native Traffic (Bytes/Macro/Cycle)
`DSP48E2`	$\sim f \cdot N_{\text{dsp}} \cdot 4 \approx 1{,}200$–$3{,}600$ B ($f \approx$ 0.1–0.3)	$7 \times$ `u64` $= 56$ B (coalesced)
`CARRY4`	$\sim f \cdot N_{\text{c4}} \cdot 4 \approx$ 10–50 B	$2 \times$ `u64` $= 16$ B
`SRLC32E`	$\sim f \cdot N_{\text{srl}} \cdot 4 \approx$ 40–180 B	$3 \times$ `u64` $= 24$ B

3.4 Projected Speed-Up

For an 8-tap DSP FIR filter with a MAC cascade ($n_{\text{chain}} = 8$, $d_{\text{macro}} = 40$, $L_{\text{aig}} = 6$):

$$T_{\text{shred}} \propto c(6 + 320) = 326c, \quad T_{\text{native}} \propto c(6) + 8 \times 2c = 22c \implies S \approx 15\times$$

Carry-heavy datapaths project $3\times$–$6\times$, while control-dominated designs project $\sim 1\times$.

3.5 Measurement Protocol

- **Setup:** Baseline (unmodified GEM) vs. Candidate (V2 engine) on identical hardware, driver, and stimulus.
- **Correctness Gate:** Every run is pre-gated with `--check-with-cpu` and 300-cycle HDL differential.
- **Timing Interval:** Encloses GPU simulation kernel only via CUDA events; excludes host setup, I/O, and compilation.
- **Sampling:** 2 warm-up runs, minimum 20 measured iterations reporting median and 95% confidence intervals.

3.6 Required Metrics and JSON Schema

```
{
  "config": {
    "build": "candidate␣|␣baseline", "design": "...", "num_blocks": B,
    "gpu": "...", "driver": "...", "cuda": "...", "git_rev": "...",
    "macro_counts": {"dsp": 0, "carry4": 0, "srlc32e": 0},
    "schedule_levels": L, "program_bytes": 0, "shared_bytes": 0
  },
  "timing": {
    "runs": [...], "median_ms": 0.0, "mean_ms": 0.0, "stdev_ms": 0.0,
    "cycles_per_s": 0.0, "macro_ops_per_s": 0.0
  },
  "nsight": {
    "sm_busy_pct": 0.0, "compute_tput_pct": 0.0, "mem_tput_pct": 0.0,
    "achieved_occupancy_pct": 0.0, "barrier_stall_pct": 0.0,
    "branch_efficiency_pct": 0.0, "shared_bank_conflicts": 0
  }
}
```

Listing 2: Benchmark JSON Output Schema

3.7 First Measured Profile (CC 7.5 Small Fixture)

3.8 Measured Results (NVIDIA GTX 1650, 14 SMs)

Table 10 details the measured performance metrics across 2,000 simulated cycles on an NVIDIA GTX 1650 GPU.

Preserving macros yields an unconditional $77\times$ reduction in graph node count ($1,995$ vs $153,604$ AIG pins). Execution speedup reaches up to $4.62\times$ on single-block workloads where the shredded graph starves GPU utilization.

Table 9: Nsight Compute Profiling Summary (Small Fixture, CC 7.5)

Metric / Property	Measured Value	Architectural Interpretation
Coalesced Access	0% excessive sectors	Field/bit-major layout delivers optimal global memory coalescing
Branch Efficiency	99.50%	Type-homogeneous queues eliminate per-instance primitive divergence
Sanitizer Checks	Clean (0 errors)	`memcheck`, `racecheck`, and `synccheck` verified clean
Program Cache	L1 92.6%, L2 100%	Single HtoD transfer remains cache-resident across cycles
Register Usage	128 regs/thread	Addressed via `-maxrregcount=180` optimization
Barrier Stall	84.4% lat. stall	Small test fixture idle; macro-dense designs fill warp-stride loops

Table 10: Part D Throughput Measurements (2,000 Cycles, Median of 5 Runs)

Design	Blocks	Shredded (V1)	Preserved (V2)	V2/V1	AIG Pins (V2/V1)
`hetero_farm` (32x each)	1	3,365 cyc/s	15,546 cyc/s	**4.62**×	1,995 / 153,604
`hetero_farm`	4	9,726 cyc/s	16,874 cyc/s	**1.73**×	(77× graph cut)
`hetero_farm`	8	17,932 cyc/s	17,046 cyc/s	**0.95**×	—
`bench_mac` (8-deep chain)	1	48,817 cyc/s	15,622 cyc/s	**0.32**×	303 / 5,286 (17×)

3.9 Threats to Validity

- **Correctness Requirement:** Unverified throughput runs failing CPU validation are strictly discarded.
- **Regime Dependence:** Performance gains depend on GPU saturation thresholds; larger GPUs extend the V2 advantage crossover point.
- **Benchmark Scale:** Profiling fixtures must provide adequate macro density (≥ 1 full wave per SM) to avoid measuring pure launch overhead.